

Advanced Lesson 10 - Exercises:

Modules, Recursion and Procedural Programming

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: A Module that averages ten integers

```
Clear[m1];  
m1 := Module[{list}, list = Table[i, {i, 1, 10}]; Mean[list]]  
  
m1
```

2: A Module that plots ten ordered pairs

```
Clear[m2];  
m2 := Module[{pairs}, pairs = Table[{i, i^2}, {i, 1, 10}];  
ListPlot[pairs, PlotStyle → PointSize[0.02`]]  
  
m2
```

3: A Module wrapping DSolve and Plot

```
Clear[f, a, y0, t, y];  
f[a_, y0_] := Module[{sol, yt}, sol = DSolve[{y'[t] == a y[t], y[0] == y0}, y[t], t];  
yt = y[t] /. First[sol];  
Plot[yt, {t, 0, 2}, PlotRange → All, PlotLabel → "a = "a", y0 = "y0"]  
  
f[2, 3]
```

```
f[-2, 3]
```

4: The same Module with y0 localised and initialised to 1

```
Clear[f, a, t, y];
f[a_] := Module[{y0 = 1, sol, yt}, sol = DSolve[{y'[t] == a y[t], y[0] == y0}, y[t], t];
  yt = y[t] /. First[sol];
  Plot[yt, {t, 0, 2}, PlotRange -> All, PlotLabel -> "a = "a", y0 = "y0"]]
f[2]
f[-2]
```

5: A Module that averages the first n integers

```
Clear[f, n];
f[n_] := Module[{list}, list = Table[i, {i, 1, n}]; Mean[list]]
{f[10], f[100]}
```

6: A linear economy, defined recursively

```
Clear[economy, a, b, n, value1];
economy[1] = value1;
economy[n_] := a economy[n - 1] + b
{economy[1], economy[2], economy[10]}
```

7: A nonlinear economy, defined through an arbitrary F

```
Clear[economy, F, X, a, b, n, value1];
F[X_] := a X + b;
economy[1] = value1;
economy[n_] := F[economy[n - 1]]
{economy[1], economy[2], economy[10]}
```

8: A two-lag economy

```
Clear[economy, F, X, Y, a, b, n, value1, value2];
F[X_, Y_] := a X + b Y;
economy[1] = value1;
economy[2] = value2;
economy[n_] := F[economy[n - 1], economy[n - 2]]
{economy[1], economy[2], economy[3], economy[5]}
```

9: The same economy, procedurally - Do, While and For

```
Clear[economyDo, economyWhile, economyFor, a, b, n, k, value1];
economyDo[n_] := Module[{e = value1}, Do[e = a e + b, {k, 2, n}];
  e]
economyWhile[n_] := Module[{e = value1, k = 1}, While[k < n, e = a e + b; k = k + 1]; e]
economyFor[n_] := Module[{e = value1, k}, For[k = 2, k ≤ n, k = k + 1, e = a e + b]; e]
{economyDo[1], economyDo[2], economyDo[10]}
{economyWhile[1], economyWhile[2], economyWhile[10]}
{economyFor[1], economyFor[2], economyFor[10]}
```

10: A four-piece function, via Which

```
Clear[f, x];
f[x_] := Which[x < -2, x2, -2 ≤ x < 1, Sin[x2], 1 ≤ x < 3, 2 x, 3 ≤ x, x]
Plot[f[x], {x, -3, 4}]
```